AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science

Faculty of Science and Technology

CSC4118: Computer Graphics

Spring 2025-2026

Section: [K], Group No: L

Project Title

Weather Explorer - An Interactive Urban Simulation with Real-Time Atmospheric and Air Quality Visualization

Supervised By

Dipta Justin Gomes

Submitted By:

Name	ID	Contribution
Mehereen Khan Mila	23-53560-3	50%
Md. Tasauf Islam	23-54025-3	50%

1. Introduction

The rapid urbanization and industrial growth of the current century have brought environmental monitoring to the forefront of global concerns. One of the most pressing issues in modern metropolitan areas is the deteriorating quality of air, often quantified by the Air Quality Index (AQI). While data is frequently available in numerical formats, there is a significant gap in how the general public perceives the tangible impact of pollution on their immediate surroundings. Our project, "Weather Explorer: A Dynamic 2D Urban Simulation with Real-Time Atmospheric and Air Quality Visualization," is a 2D interactive simulation that shows a Dhaka style city under different weather conditions. The user can change the weather, switch between two different scenes, and also connect with real weather data from the internet. It aims to bridge this gap by providing an immersive, interactive platform that visualizes the interplay between urban infrastructure and atmospheric conditions.

A primary motivation for this project is the critical environmental state of cities like Dhaka, Bangladesh. Dhaka consistently ranks among the most polluted cities globally, frequently recording "Unhealthy" or "Hazardous" AQI levels. In such high-density urban environments, the presence of particulate matter creates thick, brownish smog that reduces visibility and alters the aesthetic and physiological experience of the city. By simulating these specific conditions, ranging from the clear blue skies of a low-AQI day to the dense, suffocating haze of a polluted afternoon, this project provides a visual narrative of the "silent killer" that is air pollution.

The novelty of our project is in three parts. First, we use a local context. We made Dhaka style buildings such as apartments, a mosque with dome and minaret, and a brick kiln. We also put vehicles on the road. The background of the scenario is Dhaka's climate. Dhaka has three main seasons: summer (March to May), monsoon (June to October), and winter (November to February). The scope of this project involves creating a procedural 2D city block where environmental parameters can be toggled in real-time. Similar recent projects in the field of computer graphics have focused on hyper-realistic weather rendering or static architectural visualizations.

Each season brings its own weather. Summer brings hot days and the famous Kalboishakhi storms in the afternoon. Monsoon brings long rains and floods. Winter brings cold air with thick smog. Our three weather modes (Clear, Storm, Smog) cover the most important conditions of the city. However, Weather Explorer introduces novelty by linking functional graphics techniques, such as particle systems for rain and snowfall and fixed-function fog for smog simulation, directly to environmental data points. The overall simulation follows a day in the life of a digital city, where the user acts as an observer with the power to manipulate time and toxicity. Through the use of OpenGL and GLUT, the project demonstrates how mathematical transformations and rendering techniques can be applied to solve the real-life problem of making abstract environmental data visible and understandable to the human eye.

This project is developed using the OpenGL graphics library with the C++ programming language, providing a powerful and efficient platform for real-time three-dimensional visualization. For handling window creation, display management, and user input events, the GLUT (OpenGL Utility Toolkit) library is used. GLUT simplifies the implementation of keyboard and mouse interactions, menu controls, and the rendering loop. The methods applied in this project are based

on fundamental computer graphics principles and real-time rendering techniques. These include geometric modeling for designing buildings, roads, footpaths, and other city objects, 2D transformations for positioning and animating objects; and lighting and shading techniques for realistic illumination. Texture mapping is applied to enhance visual details of surfaces such as walls and roads. Object animation methods are used to simulate moving vehicles and environmental dynamics.

2. Related Works

The works focus on realistic rendering, environmental dynamics, and interactive visualization. Computer graphics–based city visualization and urban simulation have been widely explored in both academic research and student-level projects. The following section reviews ten relevant projects and systems that relate closely to the objectives of the proposed urban OpenGL scenario.

1. Simple City Scenery OpenGL Computer Graphics Project

This project highlights a basic 2D city environment created using OpenGL and GLUT. It includes mountains with Sun/Moon in sky, buildings, roads, one road crossing the mountain, vehicles, traffic signals, and a day–night switching feature. The primary contribution of this work lies in its structured use of OpenGL primitives to form an urban layout while maintaining simplicity for educational purposes. However, it lacks behavior-based interactions between vehicles and traffic systems.

URL: https://www.openglprojects.in/2022/07/simple-city-scenery-opengl-computer.html

2. Smart City Computer Graphics Project

The Smart City project integrates buildings, vehicles, environmental elements, and basic animations using OpenGL. It attempts to represent a modern city with emphasis on infrastructure and public utilities. Environmental elements such as trees, roads, and lighting are included, but time-based environmental transitions are limited.

URL: https://github.com/shakib04/c-plus-plus-opengl-projects

3. City View Using GLUT

This GLUT-based city view project renders a static city environment with buildings, roads, and vehicles. It demonstrates how layered drawing and color variation can create depth in a 2D scene. The project serves as a foundation for beginners but lacks interactivity and dynamic logic such as signal-controlled traffic.

URL: https://github.com/ZRSaimun/Computer_Graphics/

4. UrbanWorld – 3D Urban Environment Model

UrbanWorld is a research-based framework for generating large-scale 3D urban environments. It is designed for AI training, autonomous driving simulation, and visualization. The project demonstrates how complex urban scenes can be modeled programmatically, though it requires advanced tools beyond basic OpenGL.

URL: https://github.com/Urban-World/UrbanWorld

5. **Procedural City Generation in OpenGL**

This work focuses on generating city layouts procedurally using OpenGL. Buildings are generated algorithmically rather than manually drawn. The novelty lies in automation and scalability, allowing large city scenes to be rendered efficiently. However, the project emphasizes structure generation rather than real-time interaction or animation.

URL: https://vizworld.com/2009/05/procedural-city-screensaver-in-opengl/

6. **DAY-AND-NIGHT-CYCLE: A computer graphics and visualization mini project using OpenGL [Computer software]**

Gowda (n.d.) developed a day and night cycle simulation using OpenGL/GLUT. The scene includes a house, clouds, trees, birds, and a boat. The user can switch between day and night using the keyboard. The moon and clouds appear at night. This project inspired the day/night toggle feature in our project.

URL: https://github.com/KiranGowdaS/DAY-AND-NIGHT-CYCLE

7. **City road 3D scenario using OpenGL C++ containing buildings, street, sky, cars and day/night view**

Fh (n.d.) built a 3D city road scene using C++ and OpenGL/GLUT. The project shows buildings, a street, cars, and a sky with a day and night view. It is more advanced than our project because it uses 3D rendering, but the core idea of a city scene with a day/night cycle is the same as our project.

URL: https://github.com/marwa-fh/openGL

8. **A 2D model of the country highway scene with day, night, and rainy day modes**

Mahfuz (n.d.) created a 2D highway scene using C++ and OpenGL. The scene shows a school, a house, and a river along a highway. It supports day mode, night mode, and rainy day mode. The project also includes ambient rain sound. Like our project, it uses keyboard and mouse interaction and covers OpenGL transformation topics.

URL: https://github.com/NAI-Inc/CG-Project

9. **City-View-2D: A 2D city view simulation with weather and day/night cycle using OpenGL and GLUT**

Shuvra (2020) built a 2D city view simulation using C++ and OpenGL/GLUT in Code::Blocks. The project shows a city with buildings, vehicles, a lake, and a four-lane road. It supports day mode, night mode, and rain mode. Weather changes happen based on season. The user can control the simulation with keyboard and mouse. This project is similar to ours in terms of scene design and weather switching.

URL: https://github.com/iamshuvra/City-View-2D

10. **2D-CG-Project-Land-Sea-Breeze: CGV mini project demonstrating land and sea breeze using OpenGL/GLUT**

Stalance (n.d.) made a 2D weather simulation using C++ and OpenGL/GLUT that shows land and sea breeze during the day and at night. The scene shows how warm and cool air moves in different directions depending on the time of day. This project is similar to ours in how it visualizes atmospheric conditions visually using simple 2D primitives.

URL: https://github.com/SoniaStalance/2D-CG-Project-Land-Sea-Breeze

3. Implementation

Our project has many small features that work together. In this section we explain the key features, the main technologies we used, and the most important functions in the code by using OpenGL with GLUT in C++. The simulation represents a realistic urban scene containing roads, buildings, vehicles, mosque, natural elements, dome and minaret, and a brick kiln and a dynamic day–night cycle. The system integrates graphics rendering, animation timers, and state-based logic to create an interactive and visually appealing environment.

3.1 Key Features

The simulation has two scenes. Scene 1 is a residential area with three apartment buildings, a mosque, trees, a small pond, sidewalks, streetlights, benches, and a rickshaw which the user can drive. Scene 2 is an industrial area with a factory, a brick kiln, an electrical pylon network, a transformer, a billboard with cycling ads, a T-junction with traffic lights, and a bus driven by the user along with other background vehicles (a tanker, a pickup, and two cars).

The simulation supports three weather states. Clear weather shows a blue sky and white clouds. The Kalboishakhi storm shows a dark sky, gray clouds, slanted rain, lightning flashes, and the trees sway with the wind. Heavy smog shows a brown overlay over the whole scene and a higher AQI value. The user can press C, K, or M keys to change the weather any time.

A day and night toggle is also available with the N key. At night, the sky becomes dark blue, stars twinkle, the moon appears as a crescent shape, and the apartment windows show some lit yellow squares. Streetlights also turn on with a soft glow.

A live mode is included. When the user presses L, the program starts reading a file called weather.txt every two seconds. A separate Python script can write this file with real weather data from OpenWeatherMap API. So, the simulation can run with real Dhaka weather. If the file is missing, the simulation keeps the current state, so the program works fine offline also.

A HUD overlay in the top left shows the current state: scene name, weather, AQI, wind speed, animation speed, time of day, and live mode status. A startup screen at the beginning shows the project title and group information. The user can press S to start.

3.2 Key Technologies

We use the basic primitives of OpenGL: GL_QUADS for rectangles, GL_TRIANGLES for triangles, GL_LINES for lines (used for the pylon lattice tower), GL_LINE_STRIP for connected lines (used for lightning), GL_TRIANGLE_FAN for filled circles and the cloud shape, and GL_POINTS for stars. The coordinate system is set with gluOrtho2D(0, 1280, 0, 720), so one unit on screen is equal to one pixel.

For animation, we use glutTimerFunc with a 16 millisecond interval, which gives about 60 frames per second. The display mode is GLUT_DOUBLE for double buffering, which makes the animation smooth without flickering.

For transformations, we use glPushMatrix, glPopMatrix, glTranslatef, glRotatef, and glScalef. These are used for placing vehicles at the player position, swaying tree foliage during storms, and scaling the size of small or large trees.

For curves, we use the cubic Bezier formula. We use it to draw the cloud outlines (each cloud has three Bezier bumps on top) and to draw the sagging overhead power wires that hang between the pylons in the industrial scene.

For input, we use glutKeyboardFunc for keyboard keys and glutMouseFunc for mouse buttons. Keys a, d, c, k, m, n, l, s, 1, and 2 are used for different actions like driving, changing weather, switching scenes, and toggling modes. Mouse left and right clicks change the wind speed and animation speed.

For blending and transparency, we use glEnable(GL_BLEND) with glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA). This is used for smoke particles, the brown smog overlay, the lightning flash, and the soft halo around the sun and streetlights at night.

3.3 Key Functions

The main drawing helpers in utils.h are: drawCircle, drawDome, drawRect, bezierPoint, drawBezier, drawText, and clampf.

The weather and atmosphere functions in weather.h are: drawSky, drawSunMoon, drawStars, drawCloud, drawClouds, updateClouds, updateRain, drawRain, emitSmoke, updateSmoke, drawSmoke, triggerLightning, drawLightning, drawSmogOverlay, readWeatherFile, and initWeather.

Scene 1 functions in scene1.h are: drawGroundScene1, drawPond, drawApartment, drawMosque, drawTree, drawStreetlight, drawBench, drawRickshaw, drawCNG, drawCar1, and drawScene1.
Scene 2 functions in scene2.h are: drawGroundScene2, drawFactory, drawBrickKiln, drawTrafficLight, drawPylon, drawTransformer, drawOverheadWires, drawBillboard, drawBus, drawTanker, drawPickup, drawCarA, drawCarB, drawScene2, and emitScene2Smoke.

The main file (main.cpp) has the callback functions: display, reshape, keyboard, mouse, and timer. It also has drawStartupScreen and drawHUD for the user interface, and the main function which sets up the window and starts the GLUT main loop.

4. Screenshot of the project

Figure-1 : Start Screen

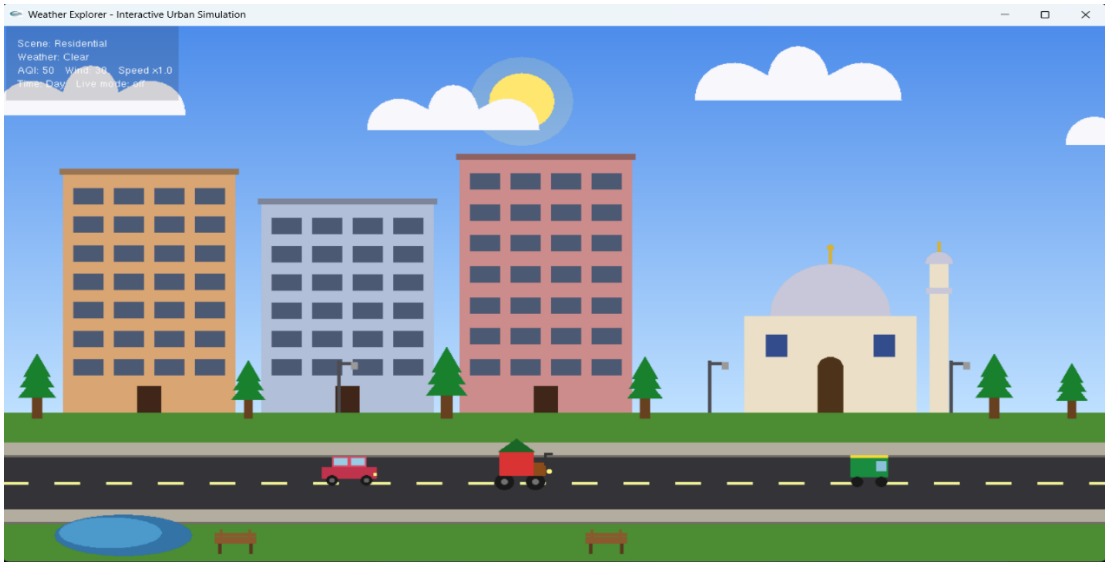

Figure-2 : Scene 1 Clear Sky

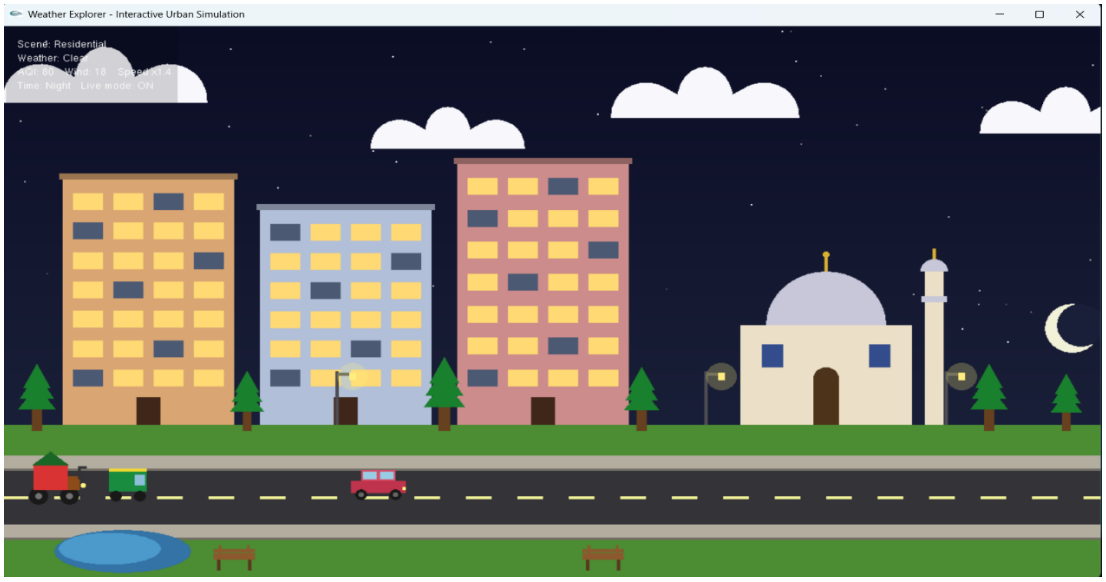

Figure-3 : Scene 1 Night Sky

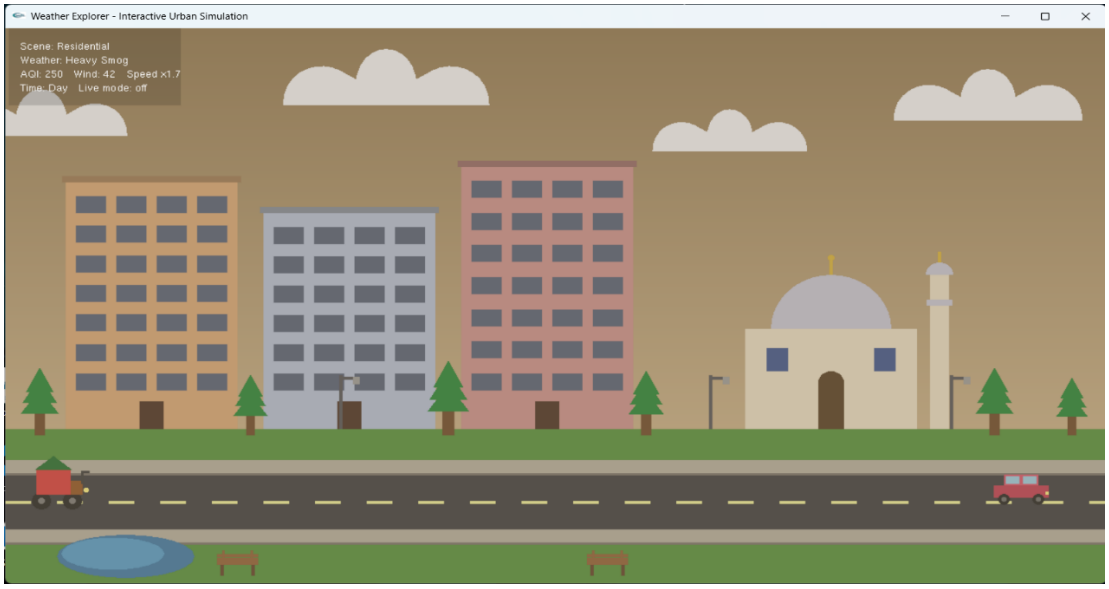

Figure-4 : Scene 1 Polluted Air

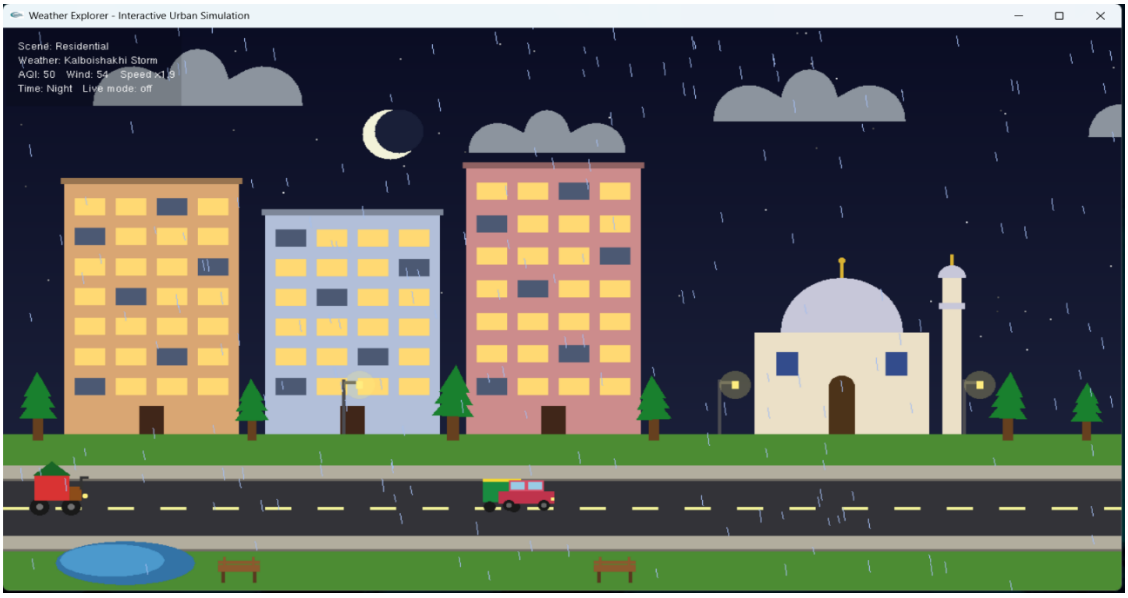

Figure-5 : Scene 1 Kalboishakhi

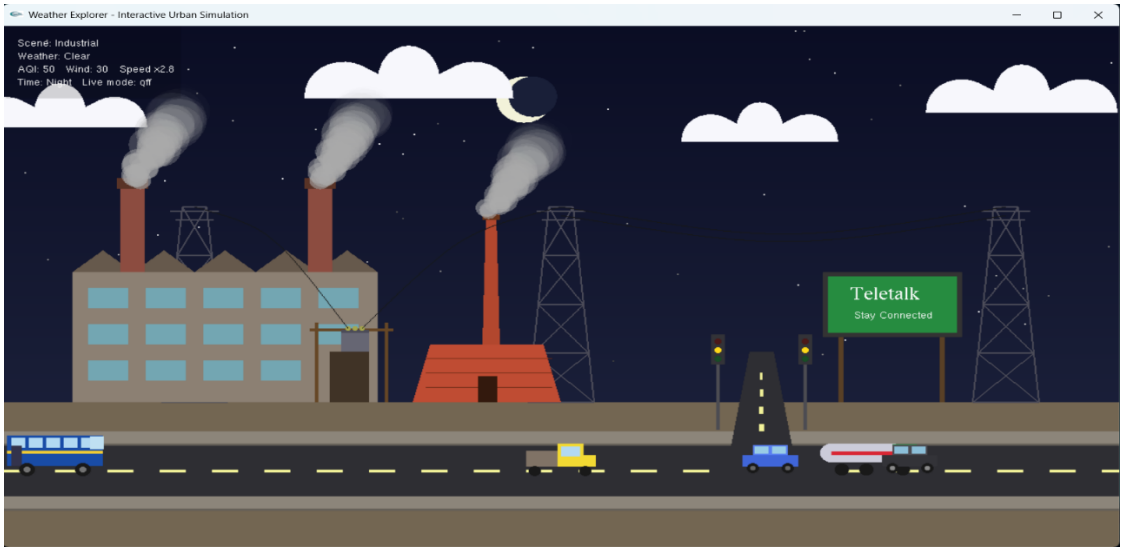

Figure-6 : Scene 2 Night Sky

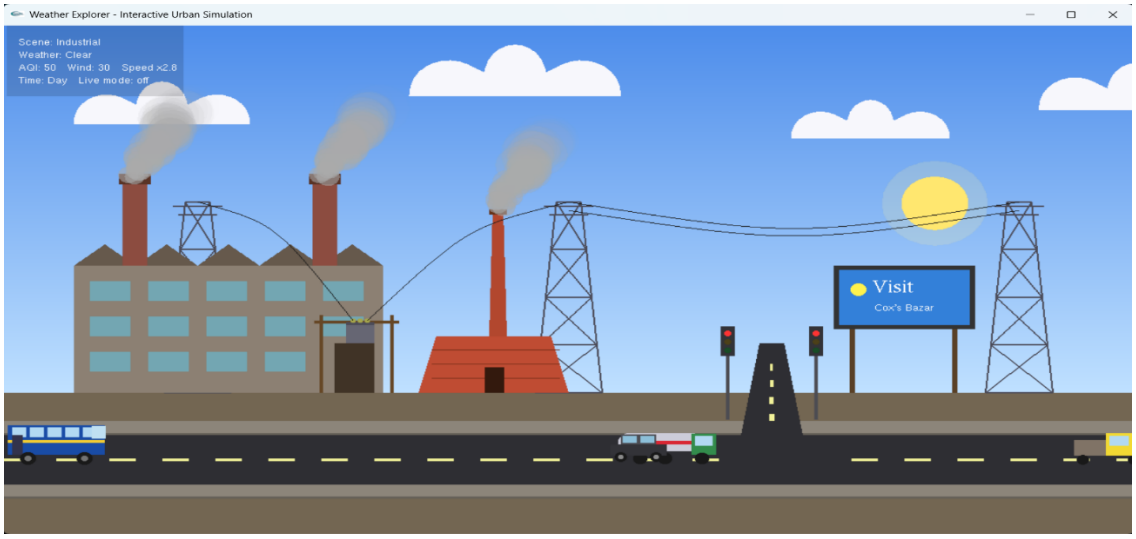

Figure-7 : Scene 2 Clear Day Sky

Figure-8 : Scene 2 Kalboishakhi

Figure-9 : Scene 2 Polluted Air

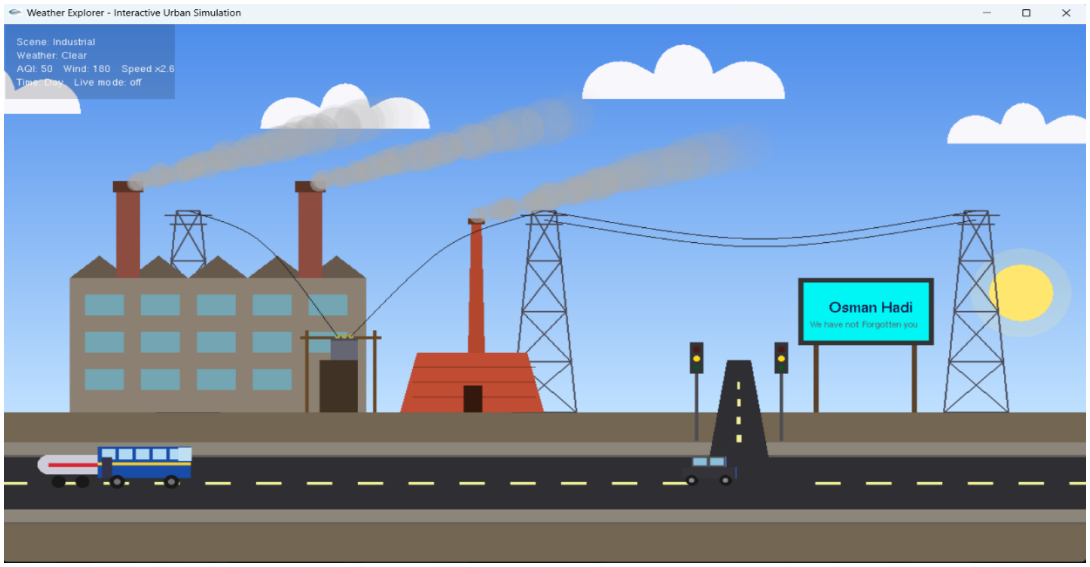

Figure-10 : Scene 2 with High Air Speed

5. Code Availability

https://github.com/tasaufBD/Computer_Graphics_FT_Project.git

6. Future Work

In the future, several enhancements can be introduced in the future to improve realism, functionality, and scalability. We want to extend this project in the following ways:

1) Convert the simulation to 3D using OpenGL 3D rendering, so the scenes have depth and the user can move the camera around.
2) Add sound effects like rain, thunder, traffic horns, and city noise to make the experience more realistic.
3) Add texture mapping support so we can show real photo images on the billboard, on the buildings, and on the ground.
4) Add more cities like Chittagong or Sylhet, each with its own buildings and local style.
5) Add pedestrians and proper traffic AI, so vehicles can stop at red lights and avoid each other.
6) Add more weather types like fog, sunset, snow, or hailstorm.
7) Port the project to Android or iOS using OpenGL ES for mobile devices.
8) Add a save and load feature, so the user can save a custom scene setup and load it later.
9) Add an air quality history graph showing AQI changes over time in live mode.
10) Improve the live mode by adding a built-in HTTP client in C++, so the Python script is not needed.

7. Conclusion

The project successfully integrates multiple graphical components such as buildings, roads, vehicles, traffic signals, and environmental elements into a single cohesive scene. Through timer-based animation and state-driven logic, the project achieves smooth movement of vehicles, realistic traffic signal behavior, and dynamic environmental transitions including day, sunset, and night modes.

The project effectively applies core computer graphics concepts, including primitive drawing, coordinating transformations, animation loops, and conditional rendering. Modular function design improves code readability and reusability, while the use of Boolean states ensures proper synchronization between traffic signals, vehicle movement, and environmental lighting. Overall, the simulation provides a clear visualization of how real-world traffic and environmental systems can be represented graphically.

Limitations of the project:

1) The project is limited to a 2D environment only. The depth, perspective and different camera angles are missing.
2) This project does not include sound features. The storm and city feel quiet even during heavy weather.
3) Physics is simplified. Rain drops fall straight, smoke goes up, but they do not interact with the buildings or the ground in a realistic way.

4) All scene objects are placed at fixed coordinates. There is no procedural generation, so the scene always looks the same. The layout and object positions are optimized for a specific window size, which may cause scaling issues on different screen resolutions.
5) The live mode needs the Python script to run separately in the background. This is an extra step for the user.
6) The billboard ads are drawn with simple shapes and text only. We did not use texture mapping, so we cannot show real photo images.
7) The vehicles drive in a straight line and do not respond to the traffic lights. They are mainly decorative.

In summary, the project covers drawing primitives, the 2D coordinate system, transformations (translate, rotate, scale), animation with the timer callback, trigonometric functions for circles and arcs, cubic Bezier curves for smooth shapes, and keyboard and mouse input handling. We also added a small extra feature to make the project more interesting and connected to the real world.

8. References

1. OpenGL Projects. (2022, July). *Simple city scenery OpenGL computer graphics project*. OpenGL Projects. https://www.openglprojects.in/2022/07/simple-city-scenery-opengl-computer.html
2. Shakib04. (n.d.). *Smart city computer graphics project* [Computer software]. GitHub. https://github.com/shakib04/c-plus-plus-opengl-projects
3. ZRSaimun. (n.d.). *City view using GLUT* [Computer software]. GitHub. https://github.com/ZRSaimun/Computer_Graphics/
4. Urban-World. (n.d.). *UrbanWorld – 3D urban environment model* [Computer software]. GitHub. https://github.com/Urban-World/UrbanWorld
5. VizWorld. (2009, May). *Procedural city generation in OpenGL*. VizWorld. https://vizworld.com/2009/05/procedural-city-screensaver-in-opengl/
6. Gowda, K. S. (n.d.). *DAY-AND-NIGHT-CYCLE: A computer graphics and visualization mini project using OpenGL* . GitHub. https://github.com/KiranGowdaS/DAY-AND-NIGHT-CYCLE
7. Fh, M. (n.d.). *OpenGL: City road 3D scenario using OpenGL C++ containing buildings, street, sky, cars and day/night view*. GitHub. https://github.com/marwa-fh/openGL
8. Mahfuz, A. (n.d.). *CG-Project: A 2D model of the country highway scene with day, night, and rainy day modes* . GitHub. https://github.com/NAI-Inc/CG-Project
9. Shuvra, I. (2020, October 5). *City-View-2D: A 2D city view simulation with weather and day/night cycle using OpenGL and GLUT*. GitHub. https://github.com/iamshuvra/City-View-2D
10. Stalance, S. (n.d.). *2D-CG-Project-Land-Sea-Breeze: CGV mini project demonstrating land and sea breeze using OpenGL/GLUT*. GitHub. https://github.com/SoniaStalance/2D-CG-Project-Land-Sea-Breeze